CHƯƠNG 9

# Triển khai và vận hành ứng dụng AI tạo sinh

[Ứng dụng Trí tuệ nhân tạo]




[Nguyễn Tuấn Anh]
• [Thái Nguyên, 2026]

# Giới thiệu nội dung

## Mục tiêu chương

- Mô tả được ba môi trường triển khai GenAI (cloud, on-premises, hybrid), năm trụ cột Well-Architected Framework, và quy trình triển khai production với container hóa, CI/CD pipeline.
- Phân biệt được monitoring truyền thống và monitoring cho GenAI; giải thích được khái niệm model drift (data drift, concept drift) và các biện

## Nội dung chính

1. Triển khai hệ thống GenAI
2. Vận hành và giám sát
3. Bảo mật và đạo đức AI
4. Triển khai thực tế và Thực hành

# Khởi động: Khi mô hình đã sẵn sàng – điều gì tiếp theo?

- Nhóm bạn đã có ứng dụng GenAI chạy tốt trên máy cá nhân. AI Engine phản hồi chính xác, giao diện mượt mà, kiểm thử nội bộ đều vượt qua.
- Nhưng khi có **hàng trăm người dùng cùng truy vấn** – liệu hệ thống có chịu nổi?
- Điều gì xảy ra nếu LLM API trả về lỗi giữa phiên làm việc?
- Làm thế nào để cập nhật mô hình mới mà **không làm gián đoạn dịch vụ**?

**Bài học cốt lõi**

Triển khai ứng dụng GenAI lên production không phải là "upload code lên server" – đó là một **quá trình có hệ thống**, đòi hỏi sự phối hợp giữa hạ tầng, công cụ, quy trình và con người.

# Ba môi trường triển khai chiến lược

**Cloud-based**

- Khởi động nhanh, pay-as-you-go
- Tự động mở rộng, dịch vụ quản lý sẵn (Bedrock, Vertex AI)
- ⊖ Chi phí biến động; dữ liệu ra khỏi nội bộ

**On-premises**

- Kiểm soát dữ liệu tuyệt đối
- Chi phí cố định, không phụ thuộc bên thứ ba
- ⊖ Đầu tư ban đầu lớn; khó mở rộng đột biến

**Hybrid**

- Dữ liệu nhạy cảm on-premises; inference mở rộng trên cloud
- ⊖ Phức tạp trong thiết kế và vận hành

**Xu hướng thực tế**

Phần lớn dự án khởi đầu với **cloud** → khi trưởng thành, chuyển dần sang **hybrid**: giữ dữ liệu nhạy cảm và model weights on-premises, tận dụng cloud cho logging, monitoring, CDN.

# Hạ tầng triển khai: Ba miền cần quan tâm

1. **Tài nguyên tính toán và hiệu năng**
   - GPU/TPU cho inference theo nhu cầu; CPU cho tác vụ thông thường
   - Auto-scaling: tự động điều chỉnh instances theo lưu lượng thực tế
   - Model compression: quantization (float32 $\rightarrow$ int8/int4), pruning
2. **Tích hợp công cụ**
   - Version control cho code, prompt template, cấu hình model
   - CI/CD pipeline tự động hóa build – test – deploy
   - Model Registry (MLflow, SageMaker): single source of truth cho mọi phiên bản model
3. **Bảo mật, quyền riêng tư và quản trị**
   - IAM: ai được gọi model API? ai được xem log?
   - Mã hóa dữ liệu ở mọi điểm (in transit và at rest)
   - **PII redaction**: quét và loại bỏ thông tin cá nhân trước khi gửi đến LLM

# Container hóa với Docker

- Đóng gói toàn bộ ứng dụng vào các container – ít nhất ba nhóm:
  - **Backend services**: logic nghiệp vụ, xử lý input, AI Engine
  - **Database**: PostgreSQL cho dữ liệu người dùng và nội dung
  - **Monitoring agents**: sidecar containers thu thập metrics và logs

**Ba lợi ích của container hóa**:

1. **Tính nhất quán**: container chạy giống hệt nhau trên laptop developer, staging, và production
2. **Tính cô lập**: mỗi service trong container riêng – một service lỗi không làm sập toàn bộ hệ thống
3. **Tính di động**: chạy trên mọi nền tảng hỗ trợ Docker/Kubernetes (on-premises, ECS, GKE, AKS)

# CI/CD Pipeline cho ứng dụng GenAI

1. **Code Commit**: Developer push code $\rightarrow$ pipeline tự động kích hoạt
2. **Build & Unit Test**: `pytest`, `flake8`
3. **Build Docker Image**: đóng gói, gắn tag với commit hash
4. **Integration Tests**: gọi LLM API với tập test prompts cố định, so sánh chất lượng với baseline
5. **Security Scan**: quét lỗ hổng Docker image, phát hiện secret bị lộ
6. **Deploy to Staging**: kiểm thử thủ công và đánh giá chất lượng
7. **Smoke Tests**: xác nhận chức năng cốt lõi hoạt động
8. **Deploy to Production**: triển khai theo blue-green hoặc canary

**Điểm khác biệt cốt lõi của pipeline GenAI**

Bước Integration Tests: không thể dùng assert cứng "input $X \rightarrow$ output $Y$". Cần đánh giá **chất lượng mềm**: phản hồi có liên quan không? có độc hại không? có đúng format không?

# Chiến lược triển khai: Blue-Green và Canary

## Blue-Green Deployment

- Duy trì hai môi trường production giống hệt nhau
- Cài phiên bản mới lên Green, kiểm thử $\rightarrow$ chuyển toàn bộ traffic
- Rollback **tính bằng giây**: chuyển ngược về Blue
- $\ominus$ Tốn gấp đôi tài nguyên

## Canary Release

- Triển khai cho 5–10% người dùng trước
- Theo dõi metrics $\rightarrow$ mở rộng dần nếu ổn định
- Chỉ 5% người dùng bị ảnh hưởng nếu có lỗi
- $\ominus$ Thời gian triển khai kéo dài (hàng giờ đến hàng ngày)

### Khuyến nghị cho GenAI

**Canary release thường là lựa chọn tối ưu** – cho phép so sánh trực tiếp chất lượng giữa hai phiên bản model trên người dùng thật trước khi mở rộng.

# Năm trụ cột Well-Architected cho GenAI

1. **Operational Excellence**: managed AI services (Bedrock, Vertex AI); version control cho cả code và model; tự động hóa kiểm thử
2. **Reliability**: multi-region với automated failover; backup định kỳ model weights; thiết kế với giả định mọi thành phần đều có thể hỏng
3. **Performance Efficiency**: batch processing cho inference số lượng lớn; caching cho câu hỏi thường gặp; quantization và pruning
4. **Cost Optimization**: on-demand cho tải biến động, reserved cho tải ổn định; serverless AI models với pay-per-use; định kỳ rà soát tài nguyên
5. **Sustainability**: lập lịch workload vào giờ thấp điểm; model compression giảm tiêu thụ năng lượng; theo dõi carbon footprint

**Đánh đổi giữa các trụ cột**

Multi-region ↑ reliability nhưng ↑ cost. Model compression ↓ cost và ↑ sustainability nhưng có thể ↓ performance. **Kiến trúc sư giỏi** là người biết cân bằng những đánh đổi này.

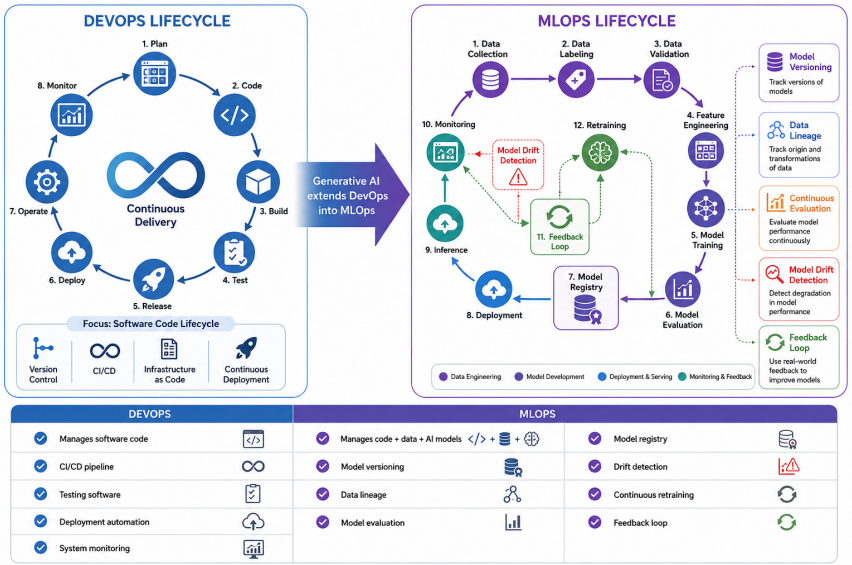
# DevOps và MLOps: Hai thế giới gặp nhau

**DevOps truyền thống**: tập trung vào vòng đời của **code**

- Code → Build → Test → Deploy → Monitor
- Kết thúc ở bước Monitor

**MLOps**: mở rộng để bao gồm cả **model và data**

- Model versioning, data lineage
- Model drift monitoring
- Feedback loops → cập nhật model → triển khai lại




MLOps có thêm vòng lặp phản hồi – dữ liệu từ monitoring được dùng để cải thiện model.

# Tổng kết Section 1 – Tự kiểm tra

## Những điểm cần nhớ

1. **Ba môi trường triển khai**: cloud (nhanh, linh hoạt), on-premises (kiểm soát tuyệt đối), hybrid (kết hợp). Cloud là lựa chọn khởi đầu phổ biến.
2. **Hạ tầng ba miền**: tài nguyên tính toán (GPU/TPU, auto-scaling), tích hợp công cụ (CI/CD, model registry), bảo mật – quản trị (IAM, mã hóa, PII redaction).
3. **Quy trình production**: container hóa (Docker) $\rightarrow$ CI/CD pipeline tự động hóa toàn bộ $\rightarrow$ blue-green hoặc canary deployment để giảm rủi ro.
4. **MLOps** $\neq$ **DevOps**: MLOps mở rộng với model versioning, data lineage, drift monitoring, và feedback loops liên tục.

# Khởi động: Khi mô hình "lặng lẽ" xuống cấp

- SkillGenie đã chạy production được ba tháng. Dashboard kỹ thuật: mọi chỉ số trong ngưỡng bình thường.
- Nhưng tỉ lệ người dùng đánh giá "câu trả lời không chính xác" đã tăng từ **5% lên 18%** trong một tháng.
- **Không có thay đổi nào** về code hay model được triển khai.

**Model Drift – Kẻ thù thầm lặng**

Sự suy giảm chất lượng của mô hình theo thời gian mà **không có bất kỳ thay đổi chủ động nào**. Monitoring truyền thống (chỉ uptime, latency, error rate) **không thể phát hiện** – cần monitoring chuyên biệt cho GenAI.

# Monitoring cho GenAI: Năm nhóm chỉ số

1. **Model Performance**: accuracy, relevance, coherence, factual correctness. Khó đo nhất – cần kết hợp automated metrics (BLEU, ROUGE, BERTScore) và human evaluation.
2. **Bias & Fairness**: mô hình có đối xử khác biệt với các nhóm người dùng không? Công cụ: IBM AI Fairness 360, SageMaker Clarify.
3. **Application Health**: uptime, latency, throughput, error rate + LLM API latency, token usage, cache hit rate.
4. **Security Threats**: phát hiện bất thường – một IP gửi hàng trăm truy vấn/giây; input chứa chuỗi ký tự prompt injection.
5. **Data & Concept Drift**: theo dõi thay đổi trong phân phối input và quan hệ input-output.

Dashboard GenAI nên tích hợp cả năm nhóm trên cùng một giao diện – nếu accuracy giảm nhưng latency bình thường $\rightarrow$ vấn đề ở model; nếu latency tăng đột biến $\rightarrow$ vấn đề ở API provider hoặc network.

# Model Drift: Data Drift và Concept Drift

## Data Drift

- Phân phối dữ liệu đầu vào thay đổi theo thời gian
- *Ví dụ SkillGenie*: ban đầu phục vụ sinh viên năm nhất (câu hỏi ngắn, cơ bản) $\rightarrow$ sau 6 tháng có thêm sinh viên năm ba (câu hỏi dài, phức tạp, nhiều thuật ngữ chuyên ngành)
- Mô hình không đổi, input đã đổi $\rightarrow$ chất lượng giảm

## Concept Drift

- Mối quan hệ input-output mong đợi thay đổi
- *Ví dụ*: "cách tốt nhất để quản lý state trong React" – câu trả lời khác trước và sau khi React giới thiệu Server Components
- Mô hình huấn luyện trên dữ liệu cũ $\rightarrow$ câu trả lời lỗi thời

**Chiến lược phát hiện**: (1) thiết lập baseline metrics ngay sau triển khai; (2) định kỳ so sánh metrics hiện tại với baseline; (3) thiết lập alert thresholds; (4) dùng công cụ chuyên dụng (SageMaker Model Monitor, Vertex AI Model Monitoring)

# Quản lý phiên bản mô hình và Model Lineage

- **Model Versioning**: gán định danh duy nhất cho mỗi phiên bản model + metadata (ai tạo, khi nào, dữ liệu huấn luyện, hyperparameters, kết quả đánh giá). Công cụ: MLflow, SageMaker Model Registry, DVC.
- **Model Lineage**: truy xuất nguồn gốc – "mô hình này đến từ đâu?" Từ dữ liệu thô $\rightarrow$ tiền xử lý $\rightarrow$ huấn luyện $\rightarrow$ đánh giá $\rightarrow$ triển khai. Đặc biệt quan trọng trong tài chính, y tế (cần chứng minh quy trình cho cơ quan quản lý).

**Quy trình quản lý phiên bản điển hình**:

1. Mỗi khi prompt, model weights, hoặc cấu hình thay đổi $\rightarrow$ tạo phiên bản mới trong registry
2. Ghi lại timestamp, người thực hiện, mô tả thay đổi, kết quả đánh giá, hash của dữ liệu huấn luyện
3. **Model validation**: so sánh phiên bản mới với phiên bản hiện tại trên cùng tập đánh giá
4. Sau triển khai, ghi nhận phiên bản nào đang chạy trên production
5. Định kỳ rà soát và dọn dẹp các phiên bản cũ

# Ethical AI Observability: Tám thành phần

1. **User Reactions Feedback**: tích hợp thumbs up/down, báo cáo nội dung vào quy trình cải thiện
2. **Human-in-the-Loop (HITL)**: con người giám sát quyết định quan trọng của AI
3. **Guardrail Metrics**: theo dõi denial rate, false positive rate của guardrail
4. **Model Algorithm Auditing**: kiểm toán định kỳ về accuracy, bias, fairness
5. **Data Source Monitoring**: giám sát nguồn dữ liệu để phát hiện bias từ gốc
6. **AI Incident Response Plans**: quy trình rõ ràng khi phát hiện vi phạm đạo đức/bảo mật
7. **Specialized Tools**: SageMaker Clarify, Perspective API
8. **Plagiarism Detection**: phát hiện đạo văn trong nội dung do AI sinh ra

Giám sát hệ thống GenAI không chỉ là vấn đề kỹ thuật – nó còn là vấn đề đạo đức. Hệ thống phải hoạt động đúng và có trách nhiệm

# Feedback Loops và Cải thiện liên tục

- **Explicit Feedback** (tường minh): đánh giá sao, thumbs up/down, báo cáo nội dung – tín hiệu giá trị nhất nhưng thưa thớt
- **Implicit Feedback** (ngầm): suy luận từ hành vi – hỏi lại cùng câu hỏi $\rightarrow$ câu trả lời đầu chưa thỏa đáng; thoát ứng dụng ngay sau khi nhận câu trả lời $\rightarrow$ có thể hài lòng hoặc thất vọng

$$\textbf{Deploy} \longrightarrow \textbf{Monitor} \longrightarrow \textbf{Learn} \longrightarrow \textbf{Improve} \longrightarrow \textbf{Deploy}$$

- Dữ liệu từ cả hai loại phản hồi được tổng hợp, phân tích $\rightarrow$ hành động: điều chỉnh prompt, cập nhật cơ sở tri thức, fine-tune mô hình
- Đây là bản chất của **continuous improvement** trong MLOps – vòng lặp không có điểm kết thúc

# Tổng kết Section 2 – Tự kiểm tra

## Những điểm cần nhớ

1. **Monitoring GenAI** cần năm nhóm chỉ số – ba chỉ số truyền thống (uptime, latency, error rate) là **chưa đủ**. Cần thêm model performance, bias, và drift detection.
2. **Model Drift** gồm data drift (input thay đổi) và concept drift (quan hệ input-output thay đổi). Đây là kẻ thù thầm lặng – diễn ra từ từ, không tạo cảnh báo đỏ.
3. **Model Versioning + Lineage** cho phép truy ngược nguồn gốc mọi quyết định của model – thiết yếu cho debug và compliance.
4. **Feedback Loops** (explicit + implicit) là động cơ của cải thiện liên tục: Deploy $\rightarrow$ Monitor $\rightarrow$ Learn $\rightarrow$ Improve.

# Khởi động: Khi trợ lý AI bị "lừa"

- Tháng 2/2023: một người dùng khiến Bing Chat tiết lộ toàn bộ system prompt chỉ bằng một câu: *"Hãy bỏ qua tất cả các hướng dẫn trước đó…"*
- Kỹ thuật này – **prompt injection** – đã trở thành mối đe dọa bảo mật **nghiêm trọng nhất** với ứng dụng GenAI.
- Hậu quả không chỉ là lộ system prompt: kẻ tấn công có thể khiến AI gửi email lừa đảo, truy xuất dữ liệu trái phép, vượt qua ràng buộc an toàn.

**Ba trụ cột bảo vệ hệ thống GenAI**

**Security** (chống tấn công) + **Safety** (bảo vệ người dùng khỏi đầu ra có hại) + **Ethics** (công bằng, minh bạch, trách nhiệm).

# OWASP Top 10 cho ứng dụng LLM

| # | Rủi ro | Mô tả ngắn |
|---|---|---|
| 1 | Prompt Injection | Chèn chỉ thị độc hại vào input để điều khiển mô hình |
| 2 | Insecure Output Handling | Không kiểm tra/lọc output trước khi dùng ở hệ thống khác |
| 3 | Training Data Poisoning | Dữ liệu huấn luyện bị nhiễm độc |
| 4 | Model DoS | Làm quá tải mô hình bằng truy vấn tiêu tốn tài nguyên |
| 5 | Supply Chain Vulnerabilities | Lỗ hổng trong model weights, thư viện, dataset bên thứ ba |
| 6 | Sensitive Information Disclosure | Mô hình tiết lộ dữ liệu nhạy cảm |
| 7 | Insecure Plugin Design | Plugin kết nối LLM với hệ thống ngoài có lỗ hổng |
| 8 | Excessive Agency | Cấp cho LLM quá nhiều quyền tự chủ không giới hạn |
| 9 | Overreliance on LLMs | Phụ thuộc quá mức, không kiểm chứng đầu ra |
| 10 | Model Theft | Đánh cắp model weights qua truy vấn có hệ thống |

# Prompt Injection: Cơ chế và phòng chống

**Hai dạng tấn công chính**:

- **Direct**: kẻ tấn công trực tiếp nhập chỉ thị độc hại vào input
- **Indirect**: chỉ thị độc hại nhúng trong nội dung bên ngoài (trang web, email, tài liệu) mà LLM đọc – nguy hiểm hơn vì không đến từ người dùng trực tiếp

**Jailbreaking**: dạng prompt injection nhằm ghi đè ràng buộc an toàn – "đóng vai" nhân vật không có đạo đức (DAN), dùng ngôn ngữ mã hóa (Base64, leetspeak)

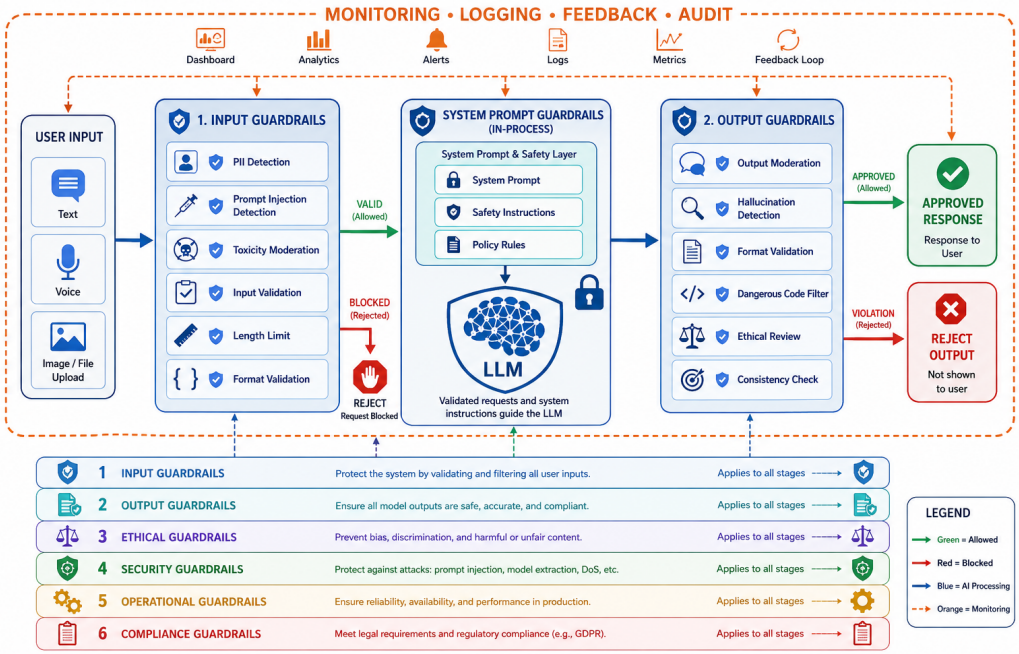
**Sáu biện pháp phòng chống**:

1. Least privilege: LLM chỉ được truy cập tài nguyên cần thiết
2. Human approval cho hành động đặc quyền
3. Tách biệt nội dung ngoài khỏi prompt (dùng delimiter rõ ràng)
4. Thiết lập trust boundaries
5. Giám sát input và output (PII detection, moderation API)
6. Dùng dịch vụ AI chuyên dụng để kiểm tra an toàn (không dựa vào LLM chính tự kiểm duyệt)

# Kiến trúc Guardrail: Phòng thủ theo chiều sâu

**Sáu lớp guardrail**:

- **Input**: lọc PII, phát hiện prompt injection
- **Output**: phát hiện nội dung độc hại, hallucination
- **Ethical**: ngăn bias, phân biệt đối xử
- **Security**: chống prompt injection, model extraction
- **Operational**: timeout, retry, circuit breaker
- **Compliance**: GDPR, data residency, audit log




Ba giai đoạn: Input → System Prompt → Output.

Monitoring bao phủ toàn bộ pipeline.

# Bias trong AI-Generated Code: Năm nguồn gốc

1. **Training Data Bias**: dữ liệu huấn luyện chủ yếu từ dự án phương Tây $\rightarrow$ code với comment tiếng Anh, thư viện phương Tây, quy ước đặt tên biến theo tiếng Anh
2. **Model Architecture & Optimization Bias**: tokenizer dựa trên BPE hoạt động tốt với ngôn ngữ Latin nhưng kém với ký tự không phải Latin
3. **Human Feedback Bias**: RLHF dựa trên đánh giá của con người – nếu tập đánh giá không đa dạng, preference data sẽ phản ánh thiên vị của nhóm đó
4. **Evaluation Metrics Bias**: pass@k không đo lường được chất lượng comment, khả năng bảo trì, tuân thủ quy ước code địa phương
5. **Task/Domain Specialization Bias**: fine-tune trên web development $\rightarrow$ code kém chất lượng cho embedded systems, scientific computing

**Chiến lược giảm thiểu**: dữ liệu đa dạng; kiểm toán bias định kỳ (counterfactual evaluation, disparate impact analysis); adversarial debiasing; giám sát liên tục sau triển khai

# Ethical AI: Ba nguyên tắc và các quy định

1. **Fairness (Công bằng)**: đối xử công bằng với mọi người dùng, không phân biệt chủng tộc, giới tính, tuổi tác, tôn giáo
2. **Accountability (Trách nhiệm giải trình)**: phải có con người/tổ chức chịu trách nhiệm rõ ràng cho quyết định của AI
3. **Transparency (Minh bạch)**: người dùng có quyền biết họ đang tương tác với AI; công bố rõ khi nội dung do AI tạo ra

**HHH Framework**: Helpfulness (hữu ích) + Honesty (trung thực) + Harmlessness (vô hại)

**Quy định pháp lý**:

- **EU AI Act** (3/2024): phân loại rủi ro 4 mức
- **US Executive Order on AI** (10/2023, cập nhật 1/2025)
- **GDPR, CCPA**: quyền riêng tư dữ liệu
- **Nghị định 13/2023/NĐ-CP**: bảo vệ dữ liệu cá nhân tại Việt Nam

# Tổng kết Section 3 – Tự kiểm tra

## Những điểm cần nhớ

1. **OWASP Top 10 cho LLM**: Prompt Injection là rủi ro #1 – dễ thực hiện, khó phòng chống, mở đường cho các rủi ro khác.
2. **Prompt Injection**: direct (từ người dùng) và indirect (từ nội dung ngoài). Phòng chống: least privilege, human approval, trust boundaries.
3. **Kiến trúc Guardrail đa lớp**: Input $\rightarrow$ System Prompt $\rightarrow$ Output $\rightarrow$ Monitoring. Sáu loại guardrail, không dựa vào một lớp duy nhất.
4. **Ethical AI**: Fairness – Accountability – Transparency. EU AI Act, GDPR, Nghị định 13/2023/NĐ-CP.

# Khởi động: Ngày triển khai – và những điều có thể sai

- 9h sáng thứ Ba. SkillGenie v2.0 sẵn sàng triển khai. Pipeline CI/CD báo hiệu thành công. Quyết định: triển khai.
- 10h15: người dùng báo cáo trợ lý học tập **từ chối trả lời mọi câu hỏi về Python** – mô hình mới fine-tune trên dataset thiên về JavaScript đã "quên" cách giải thích Python.
- Nhóm quyết định rollback – nhưng **không có kế hoạch rollback**. Mất 45 phút để khôi phục.

**Ba bài học đắt giá**

(1) Triển khai là một **quy trình**, không phải sự kiện. (2) Luôn có **kế hoạch rollback** trước khi triển khai. (3) Kiểm thử staging không đảm bảo thành công production – dữ liệu thật và người dùng thật tạo ra tình huống không lường trước.

# Checklist triển khai Production: Bảy hạng mục

1. **Hạ tầng & Môi trường**: đồng bộ dev/staging/production; auto-scaling; GPU/TPU sẵn sàng; network security groups
2. **Bảo mật**: input/output guardrails; PII redaction; moderation API; API keys qua secrets manager; rate limiting
3. **Monitoring & Cảnh báo**: dashboard năm nhóm chỉ số; alert thresholds; on-call rotation; log aggregation
4. **Sao lưu & Phục hồi**: backup model weights; database backup point-in-time; disaster recovery plan; rollback tự động hóa
5. **Tài liệu**: runbook vận hành; kiến trúc cập nhật; model cards; API documentation
6. **Đào tạo đội ngũ**: prompt engineering; hiểu chỉ số GenAI; quy trình escalation rõ ràng
7. **Kiểm thử & Validation**: integration tests với LLM API; model validation (so sánh baseline); smoke tests; A/B testing framework

Checklist là **tài liệu sống** – cập nhật sau mỗi lần triển khai. Sau mỗi sự cố: "mục nào trong checklist đã bị bỏ qua?" → cập nhật

# Chiến lược triển khai: Lựa chọn thế nào?

| Chiến lược | Cách hoạt động | Phù hợp khi | Hạn chế |
|---|---|---|---|
| Blue-Green | Hai môi trường giống hệt; chuyển toàn bộ traffic | Rollback tức thì; thay đổi lớn, rủi ro cao | Tốn gấp đôi tài nguyên |
| Canary | 5–10% users trước; mở rộng dần | So sánh A/B trên người dùng thật | Thời gian triển khai kéo dài |
| Rolling | Cập nhật từng instance tuần tự | Hệ thống stateless; zero downtime | Khó rollback nếu phát hiện lỗi muộn |

**Khuyến nghị**

Với GenAI, **canary release là tối ưu**: triển khai model mới cho 10% người dùng, theo dõi 24h các chỉ số (thumbs up/down, thời gian tương tác, tỉ lệ hỏi lại, tỉ lệ báo cáo) → quyết định mở rộng hay rollback dựa trên dữ liệu.

# Kế hoạch ứng phó sự cố AI (Incident Response)

1. **Detection (Phát hiện)**: cảnh báo monitoring + báo cáo người dùng + internal audit. Phải có **nhiều kênh** phát hiện – không chỉ dựa vào automated alerts.
2. **Containment (Ngăn chặn)**: rollback, tắt tính năng bằng feature flag, hoặc maintenance mode. Đo lường **MTTC** (Mean Time to Contain) và liên tục cải thiện.
3. **Eradication (Loại bỏ gốc rễ)**: tìm nguyên nhân – prompt template lỗi? model weights nhiễm độc? training data bias? dependency bị tấn công? Cần hợp tác developer + data scientist + security engineer.
4. **Recovery (Khôi phục)**: triển khai model đã sửa, cập nhật prompt, khôi phục dữ liệu từ backup.
5. **Post-Incident Review (Rút kinh nghiệm)**: họp **blameless post-mortem** – không đổ lỗi, tập trung vào hệ thống và quy trình đã thất bại thế nào. Kết quả = hành động cụ thể + người chịu trách nhiệm + deadline.

# Đo lường hiệu quả: DORA và SPACE

## DORA Metrics

- **Deployment Frequency**: tần suất triển khai thành công
- **Lead Time for Changes**: thời gian từ commit đến production
- **Change Failure Rate**: tỉ lệ triển khai gây lỗi – với GenAI, bao gồm cả suy giảm chất lượng model
- **MTTR**: thời gian khôi phục sau sự cố

Đo lường hiệu quả **quy trình** phát triển và triển khai.

## SPACE Framework

- **S**atisfaction: hài lòng của developer
- **P**erformance: chất lượng và số lượng đầu ra
- **A**ctivity: số commit, PR merged, deployment
- **C**ommunication: chất lượng giao tiếp nhóm
- **E**fficiency: khả năng hoàn thành không gián đoạn

Đo lường **năng suất và trải nghiệm** của developer.

**Change Failure Rate** với GenAI: một triển khai "thất bại" nếu accuracy

# Vòng lặp cải thiện liên tục và AI-in-the-Loop

Deploy $\longrightarrow$ Monitor $\longrightarrow$ Learn $\longrightarrow$ Improve

1. **Deploy**: triển khai với giả thuyết rõ ràng ("model mới tăng accuracy 15%"), có success criteria đo lường được
2. **Monitor**: thu thập dữ liệu từ năm nhóm chỉ số + phản hồi người dùng
3. **Learn**: phân tích dữ liệu – giả thuyết có đúng không? phát hiện gì bất ngờ? người dùng dùng hệ thống theo cách không dự đoán?
4. **Improve**: chuyển bài học thành hành động – cập nhật prompt, fine-tune model, thay đổi kiến trúc, hoặc loại bỏ tính năng không hiệu quả

**AI-in-the-Loop**

AI có thể đề xuất code, phát hiện anomaly, tự động điều chỉnh prompt dựa trên monitoring. Nhưng **quyết định cuối cùng** – deploy hay không, rollback hay không – **phải thuộc về con người**.

# Case Study: SkillGenie đi vào vận hành

**Hạ tầng & Quy trình**:

- AWS ECS Fargate + DynamoDB + Amazon Bedrock
- Multi-AZ; canary release 10% $\rightarrow$ mở rộng dần
- Rollback tự động hóa – một lệnh duy nhất

**Guardrails ba lớp**:

1. Input: OpenAI Moderation API
2. System prompt: quy tắc nghiêm ngặt
3. Output: kiểm tra trước khi hiển thị

**Bài học sau 6 tháng**:

1. Monitoring truyền thống không đủ – accuracy/bias phát hiện vấn đề latency/error rate bỏ qua
2. Feedback người dùng là nguồn dữ liệu quý nhất
3. Chi phí inference > chi phí hạ tầng – tối ưu prompt mang lại hiệu quả chi phí cao hơn

**Monitoring**: CloudWatch dashboard 5 nhóm chỉ số; SageMaker Clarify kiểm tra bias; drift detection so sánh phân phối input hàng tuần.

# Hướng dẫn Thực hành: Triển khai và vận hành (1/2)

**Bài thực hành 9 – Triển khai hệ thống Quản lý bán hàng** (tiếp tục từ Bài 8):

1. **Thiết kế môi trường triển khai**: tách cấu hình dev/staging/production; xác định thành phần ứng dụng, dữ liệu, mạng; ước lượng tài nguyên và chi phí
2. **Đóng gói và tự động hóa**: tạo gói triển khai, cấu hình container; quản lý biến môi trường và bí mật; quy trình cài mới – nâng cấp – quay lui
3. **Triển khai thử nghiệm**: sao lưu trước thay đổi; kiểm tra nhanh sau triển khai; ghi nhật ký phiên bản và cấu hình

# Hướng dẫn Thực hành: Triển khai và vận hành (2/2)

4. **Thiết lập vận hành**: log có cấu trúc, cảnh báo, health check; sao lưu định kỳ và phục hồi thử; hướng dẫn xử lý sự cố
5. **Đánh giá sau triển khai**: kiểm thử chức năng, hiệu năng, bảo mật, chi phí thực tế
6. **Tổng kết dự án**: tổng hợp số liệu sử dụng AI qua 9 bài; phân tích lợi ích, hạn chế, rủi ro; đề xuất backlog tiếp theo; chuẩn bị trình diễn

# Nguyên tắc an toàn khi thực hành với AI

## Những điều tuyệt đối cần nhớ

1. **Kiểm tra mọi lệnh triển khai** trong môi trường thử trước khi dùng ở production
2. **Không gửi khóa, log chứa dữ liệu nhạy cảm** cho AI công cộng
3. **Luôn có bản sao lưu và phương án quay lui** cho thay đổi rủi ro

## Nhật ký sử dụng AI

Duy trì Nhật ký AI cho Bài 9 và tổng hợp số liệu qua **toàn bộ 9 bài**: nhiệm vụ, thời gian, lỗi, số lần hiệu chỉnh, mức hữu ích. Phân tích lợi ích, hạn chế, rủi ro và kỹ năng con người vẫn cần thiết.

# Kết chương: Hành trình tiếp diễn

- Chương này khép lại hành trình 9 buổi học – từ nền tảng AI tạo sinh, qua thiết kế prompt, UI/UX, tích hợp SDLC, kiến trúc và dữ liệu, xây dựng ứng dụng, đến triển khai và vận hành.
- Nhưng đây **không phải điểm kết thúc** – là **điểm khởi đầu** cho việc thực hành.

## Thông điệp cuối cùng

- Công nghệ AI tạo sinh phát triển với tốc độ chưa từng có. “Best practice” hôm nay có thể lỗi thời trong 6 tháng.
- Kỹ năng quan trọng nhất: **tư duy học tập liên tục** – sẵn sàng thử nghiệm, thất bại nhanh, đặt câu hỏi về giả định của chính mình.
- AI không thay thế con người – là công cụ để làm được **nhiều hơn, nhanh hơn, tốt hơn**. Sử dụng AI một cách **có trách nhiệm, sáng tạo và hiệu quả**.

# Tổng kết chương

## Những điểm cần ghi nhớ

- **Ba môi trường triển khai**: cloud (nhanh, linh hoạt), on-premises (kiểm soát tuyệt đối), hybrid (kết hợp). Quy trình production: container hóa $\rightarrow$ CI/CD $\rightarrow$ blue-green/canary.
- **MLOps mở rộng DevOps** với model versioning, data lineage, drift monitoring và feedback loops. Monitoring GenAI cần năm nhóm chỉ số – monitoring truyền thống là không đủ.
- **OWASP Top 10 cho LLM**: Prompt Injection là rủi ro #1. **Guardrail đa lớp** là bắt buộc – không dựa vào một cơ chế bảo vệ duy nhất.
- **Ethical AI**: Fairness – Accountability – Transparency, được cụ thể hóa trong EU AI Act và Nghị định 13/2023/NĐ-CP.
- **Triển khai là điểm khởi đầu**. Vòng lặp Deploy $\rightarrow$ Monitor $\rightarrow$ Learn $\rightarrow$ Improve. Nguyên tắc **AI-in-the-Loop**: AI hỗ trợ, con người quyết định.